

1. **The DOM tree:** What the DOM is · Selecting elements: `getElementById`, `querySelector`, `querySelectorAll` · Loading scripts: `defer` and `DOMContentLoaded`
2. **Changing the page:** `textContent` vs `innerHTML`, and the XSS risk · Attributes, properties and form values · `classList`: styling through classes · Creating, adding and removing elements
3. **Events:** Events and `addEventListener` · The event object: `target` and `currentTarget` · `preventDefault`: forms and links
4. **Propagation and delegation:** Event propagation: capture, target and bubble · `stopPropagation`, and when not to use it · Event delegation
5. **Putting it together:** State → render: a small to-do pattern

The DOM tree

What the DOM is

The DOM (Document Object Model) is the browser's live, in-memory tree of objects built from your HTML; JavaScript reads and changes the page through it.

Example. You write `<table><tr><td>1</td></tr></table>`. In the DOM the row sits inside a `<tbody>` the browser added, so `document.querySelector('table > tr')` finds nothing, while `document.querySelector('table tr')` (any depth) works.

Common mistake. Looking for elements created by JavaScript in **View Source** and concluding the code failed. View Source shows the original file; open the **Elements** panel to see the live DOM.

Selecting elements: `getElementById`, `querySelector`, `querySelectorAll`

`document.getElementById(id)` and `document.querySelector(css)` **return one element (or null)**;
`document.querySelectorAll(css)` **returns every match as a NodeList**.

The three selection methods

Call	Argument	Returns	No match
<code>document.getElementById('save')</code>	an id, without #	one element	<code>null</code>
<code>document.querySelector('#save')</code>	any CSS selector	the first matching element	<code>null</code>
<code>document.querySelectorAll('.item')</code>	any CSS selector	a static NodeList	an empty NodeList (length 0)

Example. In a task list, `document.querySelectorAll('.tasks li.done')` finds the completed tasks, `document.querySelector('.tasks li')` the first task, and `document.querySelector('.tasks li:last-child')` the last one: the same selectors you would write in a stylesheet.

Common mistake. Passing a CSS selector to `getElementById`: `document.getElementById('#save')` looks for an element whose id is literally `#save`, returns `null`, and the next line fails with `TypeError: Cannot read properties of null (reading 'addEventListener')`.

Loading scripts: `defer` and `DOMContentLoaded`

A script can only find elements the browser has already parsed; `<script src="app.js" defer>` (or a script at the end of `<body>`) runs after the whole HTML has been read.

Where the script tag goes and what it can see

Script tag	Runs...	Can it find <code><body></code> elements?
<code><script src="app.js"></code> in <code><head></code>	immediately, blocking the parsing	No
<code><script src="app.js" defer></code> in <code><head></code>	after parsing, in order	Yes (recommended)
<code><script src="app.js"></code> just before <code></body></code>	when the parser reaches it, almost at the end	Yes, the elements above it
<code><script src="app.js" async></code>	as soon as it is downloaded	Not reliably

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
  <meta charset="UTF-8">
  <title>To-do</title>
  <link rel="stylesheet" href="styles.css">
  <script src="app.js" defer></script>
</head>
<body>
  <ul id="list"></ul>
</body>
</html>
```

Example. In the Try-it box, the first inline script runs before the paragraph exists and finds nothing; the second, placed after it, finds it; the DOMContentLoaded listener runs last, once everything has been parsed.

Common mistake. Moving the script into <head> without defer. The page shows TypeError: Cannot read properties of null (reading 'addEventListener'), not because the selector is wrong but because the element did not exist yet when the line ran.

Changing the page

textContent vs innerHTML, and the XSS risk

textContent **reads or writes plain text**; innerHTML **reads or writes HTML that the browser parses into elements, so putting untrusted text into innerHTML can run an attacker's code (XSS)**.

- innerHTML += re-parses and rebuilds **all** the existing children: listeners attached to them are lost and typed input is reset.
- innerText is similar to textContent but depends on CSS (it skips hidden text) and is slower; prefer textContent.

Example. A comment box. A visitor posts . With comment.innerHTML = text every reader of the page runs stealCookies(); with comment.textContent = text readers just see that odd text. In the Try-it box the "attack" only logs a message, but it proves the code ran.

Common mistake. Building list items with list.innerHTML += '' + input.value + ''. It opens an XSS hole (the user controls input.value) and destroys the listeners on the existing items. Use createElement + textContent + append.

Attributes, properties and form values

HTML attributes become properties of the element object (el.id, img.src, input.value, btn.disabled); getAttribute/setAttribute **work with the attribute text**, and data-* attributes hold your own data in el.dataset.

Example. For , link.getAttribute('href') is '/about' (what you wrote) while link.href is the full resolved URL such as 'https://example.com/about'. Same attribute, two views.

Common mistake. Reading input.value once when the script starts (it is still empty) and using that stale value in the click handler. Read input.value **inside** the handler, at the moment the user acts.

classList: styling through classes

el.classList **adds, removes, toggles and tests CSS classes**, so JavaScript decides **which state** an element is in and CSS decides **how** each state looks.

Example. A dark-mode button: toggleBtn.addEventListener('click', () => document.body.classList.toggle('dark'));; with all the colours defined under body.dark { ... } in the CSS.

Common mistake. Writing el.className = 'done' to mark a task. className is the whole class string, so an element with class="task urgent" loses both classes. el.classList.add('done') adds one class and keeps the others.

Creating, adding and removing elements

document.createElement(tag) **makes a new element that is not on the page yet**; append/appendChild **attach it to a**

parent so it appears, and `el.remove()` takes it out again.

```
const names = ['Ana', 'Bea', 'Carlos'];
const list = document.querySelector('#people');

for (const name of names) {
  const li = document.createElement('li'); // create
  li.textContent = name;                  // fill (safe: plain text)
  li.classList.add('person');
  list.append(li);                        // attach: now visible
}
```

Example. Rendering an array: for each name, create an `<li>`, set its `textContent`, append it to the `<ul>`. Three names produce three list items; an empty array produces an empty list, with no special case needed.

Common mistake. Creating and filling an element but never attaching it. There is no error at all; the element simply never appears. `el.isConnected` tells you whether it is in the page.

Events

Events and `addEventListener`

An event is the browser's signal that something happened (a click, a key, a form submission);

`el.addEventListener(type, handler)` **registers a function that the browser calls every time that event happens on that element.**

Events you will use most

Event	Fires when...	Typical target
click	the element is clicked (or activated with Enter/Space on a button)	buttons, links, list items
input	the value of a field changes, on every keystroke	<code><input></code> , <code><textarea></code>
change	the user commits a change (leaves the field, ticks a box, picks an option)	checkboxes, <code><select></code>
submit	a form is submitted (button click or Enter in a text field)	<code><form></code>
keydown	a key is pressed; <code>e.key</code> says which ('Enter', 'Escape', 'a')	fields, document
DOMContentLoaded	the HTML has been fully parsed	document

Example. A live character counter: `bio.addEventListener('input', () => { counter.textContent = bio.value.length + ' / 160'; });`. The handler runs on every keystroke and always reads the current value.

Common mistake. Writing `btn.addEventListener('click', handleClick());` with parentheses. `handleClick` runs once, immediately, and its return value (usually undefined) is registered as the "handler", so clicks then do nothing. The older `onclick="..."` HTML attribute is also best avoided: it mixes JavaScript into the markup and allows only one handler.

The event object: target and `currentTarget`

Every handler receives an event object describing what happened: `e.type`, `e.target` (the element where the event started), `e.currentTarget` (the element whose listener is running), `e.key` for keys, and methods such as `preventDefault()`.

- Inside a regular function handler, this is `e.currentTarget`; inside an arrow function, this is **not** the element. With arrows, use `e.currentTarget`.
- For input events, `e.target.value` is the field's current text: `console.log(e.target.value)` prints what the user has typed so far.

Example. A button with an icon: `<button data-id="7"> Delete</button>`. Clicking the icon gives `e.target = the <img>`, which has no `data-id`. `e.currentTarget.dataset.id` (listener on the button) or `e.target.closest('button').dataset.id` gives '7' wherever the user clicked.

Common mistake. Using `e.target` when you meant "the element I attached the listener to". It works while you click on bare text and breaks as soon as the element contains children (an icon, a `<strong>`, a `<span>`).

preventDefault: forms and links

Some events have a built-in browser reaction: submitting a form sends it and reloads the page, clicking a link navigates. `e.preventDefault()` **cancels that default action so your JavaScript can handle the event instead.**

```
<form id="todo-form">
  <label for="todo-input">New task</label>
  <input id="todo-input" required>
  <button>Add</button>      <!-- type="submit" by default -->
</form>

<script>
  const form = document.querySelector('#todo-form');
  const input = document.querySelector('#todo-input');

  form.addEventListener('submit', (e) => {
    e.preventDefault();          // no reload: we handle it
    const text = input.value.trim();
    if (!text) return;
    console.log('add', text);
    form.reset();                // empty the field
    input.focus();
  });
</script>
```

Example. The classic to-do form handler: `form.addEventListener("submit", (e) => { e.preventDefault(); ... })`. Remove the `preventDefault()` line and the page reloads on every submit, so the `<li>` you just appended is gone. (The Try-it box below cannot submit forms, so it shows the same idea on a link and a checkbox.)

Common mistake. Attaching a `click` listener to the submit button and forgetting `preventDefault()`: the item is added, the form submits, the page reloads, and the item vanishes. Listening to `submit` on the form is the robust choice: it covers every way of submitting and fires only after HTML validation (required) has passed, whereas the button's `click` fires before validation.

Propagation and delegation

Event propagation: capture, target and bubble

An event does not stay on the clicked element: it travels from the top of the tree down to the target (capture phase), reaches the target (target phase), and then climbs back up through every ancestor (bubble phase), running their listeners on the way.

Example. Click the button in the Try-it box: the log shows capture on `#outer` first, then the button's own listener, then bubble on `#middle` and on `#outer`. Click the section padding instead and the button is not on the path at all.

Common mistake. Believing a click belongs only to the element that was clicked. In a to-do list, a click on an item's delete button **also** reaches the item's own click listener (it bubbles), so the item is toggled as it is removed, unless your code checks what was clicked.

stopPropagation, and when not to use it

`e.stopPropagation()` **stops the event from travelling any further, so listeners on the ancestors do not run; it does not cancel the browser's default action.**

Two different methods for two different jobs

Method	Ancestors' listeners still run?	Browser default (submit, navigate, tick) still happens?
nothing	Yes	Yes
<code>e.preventDefault()</code>	Yes	No
<code>e.stopPropagation()</code>	No	Yes
both	No	No

Example. A card that opens a detail view when clicked contains a "favourite" star button. In the star's handler, `e.stopPropagation()` keeps the card from opening. The alternative is in the card's handler: `if (e.target.closest('.star')) return;`

Common mistake. Calling `stopPropagation()` to stop a link from navigating or a form from submitting. It stops the travel, not the default: the page still navigates. That job belongs to `preventDefault()`.

Event delegation

Event delegation means attaching **one listener to a common ancestor (such as the `<ul>`) and using `e.target.closest(...)` to find which child was hit, instead of one listener per child; it also covers children added later.**

```
list.addEventListener('click', (e) => {
  const deleteBtn = e.target.closest('.delete');
  if (deleteBtn) {
    deleteBtn.closest('li').remove();
    return; // do not also toggle
  }
  const item = e.target.closest('li');
  if (!item) return; // click on the list padding
  item.classList.toggle('done');
});
```

Example. In the Try-it box, add three new items with the button, then click and delete them: they work immediately, although no listener was ever attached to them. The only listener is the one on the `<ul>`.

Common mistake. Using `e.target` directly instead of `closest`. If an item is `<li><span>Buy milk</span> ...</li>`, a click on the text gives `e.target = the <span>`, so `e.target.classList.toggle('done')` styles the span (or the wrong element), and `e.target.dataset.id` is undefined.

Putting it together

State → render: a small to-do pattern

Keep the data in a JavaScript array (the **state), let event handlers change only the state, and call one `render()` function that rebuilds the list from the state after every change.**

- Clean the input: `input.value.trim()`, and ignore empty text.
- Accessibility: a clickable `<li>` cannot be reached with the Tab key. Put the toggle on a real `<button>` (or a checkbox) inside the item, so keyboard users can use it too.
- In your own page, use a `<form>` with a submit listener and `preventDefault()` (see that card). The sandboxed preview below cannot submit forms, so it listens to the button and to the Enter key directly.

```
const todos = []; // state
let nextId = 1;

function render() {
  list.replaceChildren(); // start from empty
  for (const todo of todos) {
    const li = document.createElement('li');
    li.dataset.id = todo.id;
    li.classList.toggle('done', todo.done);
    const text = document.createElement('button');
    text.className = 'toggle';
    text.textContent = todo.text; // never innerHTML
    const del = document.createElement('button');
    del.className = 'delete';
    del.textContent = 'x';
    del.setAttribute('aria-label', 'Delete ' + todo.text);
    li.append(text, del);
    list.append(li);
  }
}
```

```
}  
}
```

Example. Adding "Buy milk" pushes { id: 1, text: 'Buy milk', done: false } and renders one item; clicking it flips done to true and renders again (now crossed out); clicking x filters it out of the array and renders an empty list. At every moment todos describes exactly what is on screen.

Common mistake. Updating the DOM and the array separately. Remove the but forget to remove the array item, and it reappears the next time render() runs. With state → render, handlers never touch the DOM directly; they change the state and call render().